Comment marche gameboy3

La documentation technique — écrite pour quelqu'un qui découvre le code. Chaque mot nouveau est expliqué, et le lexique, à la fin, les reprend tous.

1. Les bases — si tu n'as jamais programmé
2. A. Ce que contient le dossier du projet
3. B. Du texte à l'écran : suivre un vrai programme
4. C. Le compilateur, fichier par fichier
5. D. L'atelier : la page où l'on travaille
6. E. Les leçons et le parcours
7. F. Les vérifications : comment on sait que tout marche
8. G. Ajouter quelque chose : un jeu, une leçon, une image, une fonction
9. H. Enregistrer et partager : git
10. Le lexique

Les bases — si tu n'as jamais programmé

Ces bases sont les mêmes que celles du cours complet. Si tu les as déjà lues, passe directement à la partie A.

0. Avant tout : ce document est fait pour toi

Tu n'as jamais programmé ? C'est exactement pour toi. Ce document ne suppose rien : chaque mot nouveau est expliqué la première fois qu'il apparaît, et il est aussi dans le **lexique**, à la fin, rangé par ordre alphabétique.

Comment le lire. Lis « Les bases » dans l'ordre, une partie à la fois, sans te presser. Ensuite, chaque leçon a deux encadrés pour t'aider : **« Les mots de cette leçon »** (les mots techniques du programme, expliqués) et **« Le programme, ligne par ligne »** (chaque ligne du programme dite en français).

Le plus important : essayer. On n'apprend pas à programmer en lisant seulement, comme on n'apprend pas le vélo dans un livre. Ouvre l'atelier, tape le programme de la leçon, appuie sur « Compiler et lancer », regarde ce qui se passe, puis **change un nombre** et relance. Se tromper fait partie du jeu : la console ne casse jamais.

1. Un ordinateur suit des instructions, une à une

Un ordinateur ne réfléchit pas : il obéit. Il fait exactement ce qu'on lui dit, dans l'ordre, très vite — des millions de fois par seconde — mais sans jamais deviner ce qu'on voulait dire.

Comme une recette de cuisine. « Casser deux œufs. Ajouter le sucre. Mélanger. » Chaque phrase est une **instruction** ; on les fait l'une après l'autre, de la première à la dernière. Un programme, c'est pareil : une liste d'instructions, écrites dans une langue que la machine comprend.

Il faut tout dire. Une personne comprend « mets un peu de sel ». Un ordinateur, non : il faut dire combien, où, et quand. C'est pour ça que le code paraît précis et un peu raide : il ne laisse rien au hasard.

2. La Game Boy : une petite console de 1989

La Game Boy est une console de jeux portable, sortie en 1989. On y glissait une **cartouche** — un petit boîtier de plastique qui contient un jeu. La **Game Boy Color** (1998) est la même console, avec des couleurs.

Son écran fait 160 pixels de large et 144 de haut. Un **pixel** est un tout petit carré de couleur : l'image entière est faite de 160 × 144 = 23 040 de ces carrés, comme une mosaïque.

Elle est très lente comparée à un téléphone d'aujourd'hui — des milliers de fois. C'est ce qui rend la programmation intéressante : chaque calcul compte, et on voit vraiment ce que fait la machine.

Tu n'as pas besoin d'une vraie Game Boy. L'atelier en contient une **imitée** dans le navigateur (on dit un **émulateur**) : elle fait exactement ce que ferait la vraie. La cartouche que tu fabriques marcherait aussi sur une vraie console.

3. Un programme, c'est un texte

Un programme s'écrit comme un texte, avec des lettres, des chiffres et quelques signes : `( ) { } ; = " "`. Ici, on l'écrit dans le langage **C++** (on dit « C plus plus »), un langage très utilisé pour les jeux vidéo.

Un langage de programmation a des règles strictes, comme l'orthographe, mais plus sévères : un signe oublié et la machine ne comprend plus. Pas de panique : l'atelier dit **où** est l'erreur et **quoi** corriger.

Le plus petit programme possible : il ne fait rien, mais il est juste.

```
int main() {
  while (true) {
    image();
  }
}
```

`int main()` veut dire « ici commence le programme » ; `while (true)` veut dire « recommence sans fin » ; `image()` veut dire « attends l'image suivante ». Les **accolades** `{ }` entourent ce qui va ensemble. Tout cela est expliqué dans les parties suivantes.

4. Le compilateur : le traducteur

La console ne lit pas le C++. Elle ne comprend que des **nombres** : des suites d'**octets** (un octet est un nombre de 0 à 255). Il faut donc **traduire** le texte en nombres.

Le compilateur est ce traducteur. Quand tu appuies sur « Compiler et lancer », il lit ton programme du début à la fin, vérifie qu'il respecte les règles, puis le transforme en octets — environ 32 000 — qui forment une **cartouche**.

S'il trouve une erreur, il s'arrête et le dit : le numéro de la ligne, et ce qui ne va pas, en français. Par exemple : « ligne 5 : il manque un point-virgule ». C'est normal et utile : il vaut mieux une erreur dite tout de suite qu'un jeu qui fait n'importe quoi.

5. L'atelier : là où l'on écrit et où l'on joue

L'atelier est la page où tout se passe (index.html, ouverte par lancer.bat). À gauche : le **programme**, que tu écris. À droite : la **console**, qui joue la cartouche.

Le bouton « ▶ Compiler et lancer » (ou Ctrl + Entrée) traduit le programme et le lance aussitôt dans la console. En dessous, la page dit ce que la cartouche contient, ou l'erreur s'il y en a une.

Les touches du clavier font la manette : les flèches pour la croix, X pour le bouton A, Z pour B, Entrée pour START.

Les modes, en haut : 🎨 CRÉATION (écrire, dessiner, composer de la musique), 📚 APPRENDRE (les leçons, dans l'ordre), 🔎 MACHINE (ce que la console exécute vraiment), 🔗 LES SOURCES (tous les documents).

6. Lire une ligne de code

On lit de haut en bas, et chaque ligne de gauche à droite — comme un livre. La console exécute les instructions dans cet ordre.

Le point-virgule `;` termine une instruction, comme le point termine une phrase. L'oublier est l'erreur la plus fréquente ; le compilateur la signale.

Les accolades `{ }` regroupent des lignes qui vont ensemble : tout ce qui est entre `{` et `}` forme un **bloc**. Les lignes d'un bloc sont décalées vers la droite (on dit **indentées**) pour qu'on voie d'un coup d'œil ce qui est dedans. Le décalage n'est que pour l'œil : la console ne le regarde pas.

Les commentaires commencent par `//` : tout ce qui suit, jusqu'au bout de la ligne, est pour l'humain qui lit, et la console l'ignore. Ils expliquent ce que fait le code.

```
texte(1, 2, "BONJOUR");   // écrit BONJOUR en haut à gauche
```

Les majuscules comptent : `score` et `Score` sont deux noms différents. Et les noms n'ont ni espace ni accent : on écrit `meilleurScore`, pas « meilleur score ».

7. Écrire à l'écran : les coordonnées

Pour écrire, il faut dire OÙ. L'écran est découpé en une grille de **20 colonnes** et **18 lignes** ; chaque case fait 8 × 8 pixels et peut contenir une lettre.

La case (0, 0) est en haut à gauche. La première coordonnée est la **colonne** (vers la droite), la seconde la **ligne** (vers le bas). Attention : on compte à partir de **zéro**, pas de un. La colonne 19 est la dernière à droite, la ligne 17 la dernière en bas.

```
texte(5, 8, "SALUT");   // colonne 5, ligne 8 : à peu près au milieu
```

Des parenthèses après un nom, c'est un appel : `texte(5, 8, "SALUT")` demande à la console d'exécuter « texte » avec trois **arguments** — les valeurs entre parenthèses, séparées par des virgules. Le mot entre guillemets `"…"` est un **texte** tel quel.

Au pixel près, pour ce qui bouge : 160 pixels de large (0 à 159), 144 de haut (0 à 143). Les lutins (les personnages) se placent ainsi.

8. Retenir un nombre : la variable

Une variable est une boîte avec un nom, dans la mémoire de la console, qui retient un nombre. On peut lire ce nombre, et le changer.

```
uint8_t score = 0;   // une boîte « score », qui commence à 0
```

`uint8_t` veut dire « un nombre entier de **0 à 255** » (un **octet**). C'est le seul genre de nombre de cette console. Au-delà de 255, il **repart à 0**, comme le compteur kilométrique d'une vieille voiture qui repasse à zéro.

Le signe `=` **veut dire « prend la valeur »**, pas « est égal ». `score = score + 1;` se lit : « score prend la valeur de score plus un » — il augmente de 1. On l'écrit aussi `score++;` ou `score += 1;`.

Une constante (`const uint8_t VIES = 3;`) est une variable qui ne change jamais : on lui donne un nom pour que le programme se lise mieux. On les écrit souvent en MAJUSCULES.

Un tableau (`uint8_t cases[10];`) est une rangée de boîtes sous un même nom : `cases[0]`, `cases[1]` … `cases[9]`. Le nombre entre crochets dit **laquelle** ; on commence à 0.

9. La boucle du jeu : 60 images par seconde

Un jeu, c'est un dessin animé. La console montre **60 images par seconde** ; entre deux images, le programme regarde la manette, déplace les personnages, et prépare l'image suivante.

```
while (true) {
  image();          // attendre l'image suivante
```

```
  // … ce qui se passe à chaque image
}
```

`while (true) { … }` répète ce qui est entre les accolades **sans jamais s'arrêter** : c'est la **boucle du jeu**. `image()` attend l'image suivante : sans elle, le programme irait trop vite et tout serait fini avant qu'on voie quoi que ce soit.

Ce qui est avant la boucle se fait une seule fois (préparer l'écran, mettre le score à zéro). Ce qui est dedans se fait 60 fois par seconde.

10. Décider : « si… sinon… »

`if` **veut dire « si »** : la console teste une **condition** ; si elle est vraie, elle fait ce qui est dans les accolades, sinon elle passe.

```
if (bouton(A)) {        // si le bouton A est enfoncé…
  texte(1, 1, "BRAVO");  // … écrire BRAVO
} else {                // sinon…
  texte(1, 1, "     ");  // … effacer
}
```

Comparer : `==` « est égal à » (deux signes `=`, pour ne pas confondre avec « prend la valeur »), `!=` « est différent de », `<` « plus petit que », `>` « plus grand que », `<=` et `>=` « ou égal ».

Combiner : `&&` veut dire « et » (les deux doivent être vrais), `||` veut dire « ou » (l'un des deux suffit), `!` veut dire « pas ».

11. Répéter : les boucles qui comptent

`for` **répète un nombre de fois connu.** On lui donne un compteur, une condition pour continuer, et ce qu'on fait au compteur après chaque tour.

```
for (uint8_t i = 0; i < 5; i++) {   // i vaut 0, 1, 2, 3, 4
  texte(i, 3, "X");                  // un X dans les colonnes 0 à 4
}
```

`while (condition)` **répète tant qu'une condition est vraie** — on ne sait pas d'avance combien de fois. `break;` sort de la boucle tout de suite.

12. Ranger des gestes : les fonctions

Une fonction est un groupe d'instructions qui a un nom. On l'écrit une fois, puis on la lance autant de fois qu'on veut en écrivant son nom suivi de parenthèses.

```
void effacerLigne(uint8_t ligne) {       // une fonction à nous
  texte(0, ligne, "                    ");
}

effacerLigne(4);                          // on la lance pour la ligne 4
```

`void` dit que la fonction **ne rend rien** ; `uint8_t nom(…)` dit qu'elle **rend un nombre**, avec `return`. Ce qui est entre les parenthèses de sa définition (`uint8_t ligne`) sont ses **paramètres** : des boîtes qui reçoivent les valeurs données à l'appel.

La console a ses propres fonctions toutes prêtes : `texte`, `poser`, `sprite`, `bouton`, `image`… Ce sont elles qui touchent l'écran, la manette et le son.

13. « #include » : demander une fonction de la console

La cartouche est petite (32 768 octets pour tout le jeu). La console ne met donc dans ta cartouche **que les fonctions que tu demandes**.

```
#include <texte>    // « ce programme se sert de texte() »
```

Une ligne `#include <nom>` **par fonction employée**, en haut du programme. Sans elle, le compilateur refuse et dit laquelle ajouter. Une ligne de trop ne coûte rien.

`#include "fichier.cpp"` (avec des guillemets) est différent : il verse **ton propre fichier** à cet endroit, comme si son texte était écrit là. Ça permet de ranger un grand programme en plusieurs onglets.

14. Les dessins : pixels, tuiles et lutins

Une tuile est un petit dessin de 8 × 8 pixels. Le décor (le fond de l'écran) est une grille de tuiles, comme un carrelage. On l'écrit avec des signes, une rangée par ligne :

```
Tuile MUR = {
  "########",
  "#++#+++#",
  …
};
```

Chaque signe est un pixel : « . » le plus clair, « - » clair, « + » foncé, « # » le plus foncé. (On peut aussi écrire les chiffres 0, 1, 2, 3.) En couleur, ces quatre nuances deviennent les quatre couleurs d'une **palette**.

Un Perso est un dessin de 16 × 16 (ou 32 × 32) : quatre tuiles (ou seize) en carré.

Un lutin (en anglais « sprite ») est un dessin qui bouge librement, au pixel près, par-dessus le décor : le héros, les ennemis, une balle. `poser()` met une tuile dans le décor ; `sprite()` place un lutin.

Dessiner une fois, agrandir comme on veut : un nombre en plus à la fin — `sprite16(0, 40, 40, HEROS, 3)` — montre le dessin trois fois plus grand ; le compilateur le redessine, il n'y a toujours qu'un seul dessin.

15. Quand ça ne marche pas : les erreurs

Une erreur n'est pas un échec, c'est une information. Tous les programmeurs en font, tout le temps.

Lis le message : il donne la **ligne** et la **raison**. Va à cette ligne (le numéro est à gauche du programme) et regarde juste avant aussi : un point-virgule oublié à la ligne 4 est souvent signalé à la ligne 5.

Les erreurs les plus courantes : un `;` oublié ; une accolade `}` en trop ou en moins ; un nom mal écrit (`Texte` au lieu de `texte`) ; un `#include` oublié ; un texte sans ses guillemets.

Avance par petits pas : change une seule chose, relance, regarde. Si ça casse, tu sais exactement ce qui a cassé. Et Ctrl + Z annule la dernière modification.

A. Ce que contient le dossier du projet

Le projet est un dossier (on dit aussi un répertoire) : un classeur dans ton ordinateur, qui contient des **fichiers** et d'autres dossiers. Chaque fichier a un nom et une **extension** — les lettres après le point — qui dit son genre :

- `.cpp` : un programme écrit en C++ (les jeux, les exemples) ;

- `.js` et `.mjs` : un programme écrit en **JavaScript**, le langage dans lequel l'atelier, le compilateur et l'émulateur sont écrits ;

- `.html` : une page web (l'atelier est une page web, ouverte dans le navigateur) ;

- `.gb` et `.gbc` : une cartouche (le jeu compilé, que la console joue) ;

- `.md` : un texte simple (les documents comme LISEZMOI.md) ; `.pdf` : un document à lire ou imprimer.

Voici le dossier, tel qu'il est en ce moment. La deuxième colonne dit à quoi sert chaque élément ; pour un fichier, c'est la première phrase de son propre commentaire, lue dans le fichier lui-même.

`aide-fonctions.js`	La fiche de chaque fonction de la console : ses arguments, et ce qu'elle fait.	106 lignes
`analyse-rom.js`	Ce que la cartouche occupe : la ROM, la mémoire de travail, et ce qui pèse.	290 lignes
`animations-tuiles.js`	Les tuiles animées : l'eau qui ondule, le feu qui danse, l'herbe qui plie.	175 lignes
`barre-paint.js`	La barre de couleurs, comme dans Paint — avec les HUIT PALETTES du jeu (0 à 7).	266 lignes
`bibliotheque.js`	Des dessins tout faits, à prendre.	3168 lignes
`cartouches/`	des cartouches du commerce, pour l'émulateur (jamais publiées : droits de Nintendo)	2 éléments
`compilateur/`	le compilateur : ce qui transforme ton texte C++ en cartouche	11 éléments
`cours/`	un PDF par leçon du cours de programmation	161 éléments
`cours.html`	L'ancienne page du cours mène au parcours (tuto.html), et à la même étape :	41 lignes
`demarrer.mjs`	Démarrer l'atelier, sur n'importe quel PC — sans XAMPP, sans configuration.	339 lignes
`desassembleur.js`	Relire une cartouche : des octets aux instructions.	306 lignes
`documents/`	les documents : le cours complet, cette documentation, l'historique des changements	16 éléments
`editeur-airs.js`	Écrire une mélodie à la souris, plutôt que de taper des noms de notes.	726 lignes
`editeur-carte.js`	Dessiner tout l'écran d'un coup, au pixel — une méga-tuile, par son nom.	2856 lignes
`editeur-code.js`	L'éditeur de code, à la manière de VS Code : les numéros de ligne, les	988 lignes
`editeur-couleurs.js`	L'atelier des couleurs : huit palettes, à la souris.	792 lignes
`editeur-scene.js`	Les scènes jouables : une carte, ses murs, un joueur, des acteurs, des	1518 lignes
`editeur-tuiles.js`	Dessiner ses tuiles à la souris, plutôt que de taper des chiffres.	1885 lignes
`emulateur.js`	Registres, tels qu'ils ont été écrits.	1980 lignes
`etat-console.js`	Sauver l'état de la console, et y revenir — au cycle près.	71 lignes
`exemples/`	des programmes tout faits (et leurs cartouches), ouverts par le menu des exemples	91 éléments
`exporter-html.js`	Exporter la cartouche courante en une page HTML autonome.	160 lignes
`files/`	un dossier du projet	1 éléments
`garde.js`	garde.js — dire POURQUOI une page ne démarre pas.	233 lignes
`images/`	des captures d'écran de l'atelier	8 éléments

`importer-gbtd.js`	Faire entrer les tuiles d'un fichier « .gbr » — celui du logiciel Game Boy	190 lignes
`importer-image.js`	Faire entrer une image dans le programme, sous forme de tuile.	251 lignes
`importer-son.js`	Importer de la musique : un fichier MIDI, ou un son enregistré (WAV).	186 lignes
`index.html`	Ce que les réglages pilotent.	6973 lignes
`inspecteur.js`	L'inspecteur : regarder DANS la console pendant qu'elle tourne.	235 lignes
`lancer.bat`	—	84 lignes
`livrets/`	un PDF par leçon, étape par étape	1370 éléments
`manette.js`	Jouer autrement qu'au clavier : une vraie manette, ou l'écran tactile.	151 lignes
`modeles-jeux.js`	Les modèles de jeux : un jeu entier, prêt à jouer — et à changer.	358 lignes
`nuancier.js`	Une seule palette pour tout, et la console qui suit.	271 lignes
`outils/`	des programmes qu'on lance en ligne de commande : compiler, fabriquer les PDF…	19 éléments
`package.json`	—	41 lignes
`programme.js`	Où poser un morceau dans le programme.	59 lignes
`projets/`	tes projets : un dossier par jeu, ouvert par « 📂 Ouvrir »	8 éléments
`projets-serveur.mjs`	Les projets, sur le disque — servis par Node, sans PHP.	278 lignes
`projets.js`	Les projets, vus de la page : un lecteur, et un bandeau qui dit où l'on est.	461 lignes
`projets.php`	Les projets, sur le disque.	421 lignes
`question-include.js`	« Il manque un #include : l'ajouter ? » — la question douce.	94 lignes
`ranger-personnages.js`	Les personnages, un fichier chacun : le parent et ses enfants.	262 lignes
`recherche-globale.js`	Tout chercher, d'un seul endroit : Ctrl+K dans l'atelier.	175 lignes
`recherche-page.js`	recherche-page.js — la recherche des pages de documentation.	128 lignes
`recherche.js`	Chercher, partout de la même façon.	347 lignes
`sommaire.html`	La recherche dans le sommaire : un script ordinaire, qui marche même ouvert par un double-clic.	165 lignes
`sources.js`	L'onglet « Les sources » : tous les documents du projet, d'un clic.	163 lignes
`supprimer.js`	Supprimer un élément du jeu — seulement s'il ne sert nulle part.	201 lignes
`tout-le-jeu.js`	L'onglet « 📦 Tout le jeu » : chaque élément du programme, rangé par groupe.	444 lignes
`tuto/`	les leçons du parcours, et les outils qui les font tourner	12 éléments
`tuto.html`	--- le sommaire ---	1218 lignes
`tutoriels/`	les cartouches et les captures des leçons	660 éléments
`verification/`	les vérifications : des programmes qui contrôlent que tout marche	27 éléments
`verrous.js`	Verrouiller un dessin : on s'en sert, on ne le modifie plus.	111 lignes
`versions.js`	Les versions du programme : des photos datées, qu'on peut rouvrir.	139 lignes
`vue-machine.js`	Voir ce que la console exécute vraiment.	132 lignes

À retenir

Trois choses font tout le projet : **l'atelier** (index.html, la page où l'on écrit et où l'on joue), **le compilateur** (le dossier compilateur/, qui traduit le texte en cartouche) et **l'émulateur** (emulateur.js, la Game Boy imitée qui joue la cartouche). Tout le reste les aide : les leçons, les exemples, les outils, les vérifications, les documents.

B. Du texte à l’écran : suivre un vrai programme

Que se passe-t-il quand on appuie sur « Compiler et lancer » ? On va le voir en suivant un vrai programme, `exemples/bonjour.cpp`, à travers chaque étape. Tout ce qui est montré ici a été **calculé** par le vrai compilateur et le vrai émulateur au moment où ce document a été fabriqué.

L’idée générale, comme une chaîne de fabrication : le texte entre d’un côté ; chaque étape le transforme un peu ; une cartouche sort de l’autre côté, et la console la joue.

```
ton texte (.cpp)
   │  1. rassembler   — verser les fichiers #include "…" au bon endroit
   ▼
un seul texte
   │  2. analyser     — lire les mots et ranger le programme en arbre
   ▼
un arbre
   │  3. compiler     — transformer l’arbre en instructions du processeur
   ▼
des instructions (des octets)
   │  4. fabriquer    — ajouter l’en-tête et les dessins : la cartouche
   ▼
une cartouche de 32 768 octets
   │  5. jouer        — l’émulateur exécute la cartouche, 60 images par seconde
   ▼
l’écran
```

Le programme de départ

Le voici, tel qu’il est dans le fichier :

```
// Une cartouche Game Boy, écrite en C++.
//
// Ce fichier est du vrai C++ : votre éditeur le colore, et il se lit comme
// n'importe quel autre. Il n'est pas compilé par g++ — il est traduit en code
// machine SM83 et gravé dans une cartouche de 32 Ko qui démarre sur une
// console d'origine.
//
//   node gb3.mjs exemples/bonjour.cpp

#include <texte>    // écrit un texte à l’écran
#include <bouton>   // lit un bouton de la manette

int main() {
  texte(6, 4, "BONJOUR");
  texte(3, 8, "APPUIE SUR A");

  uint8_t compte = 0;

  while (true) {
    image(); // attend l'image suivante : soixante fois par seconde

    if (bouton(A)) {
      texte(3, 8, "BRAVO       ");
      compte++;
    }

    if (bouton(B)) {
      texte(3, 8, "APPUIE SUR A");
    }
```

```
  }

  return 0;
}
```

🔍 bonjour.cpp, ligne par ligne — chaque ligne dite en français

`// Une cartouche Game Boy, écrite en C++.`	Un commentaire : une phrase pour celui qui lit (« Une cartouche Game Boy, écrite en C++. »). La console l'ignore.
`// Ce fichier est du vrai C++ : votre éditeur le colore, et il se lit comme`	Un commentaire : une phrase pour celui qui lit (« Ce fichier est du vrai C++ : votre éditeur le colore, et il se lit comme »). La console l'ignore.
`// n'importe quel autre. Il n'est pas compilé par g++ — il est traduit en code`	Un commentaire : une phrase pour celui qui lit (« n'importe quel autre. Il n'est pas compilé par g++ — il est traduit en code »). La console l'ignore.
`// machine SM83 et gravé dans une cartouche de 32 Ko qui démarre sur une`	Un commentaire : une phrase pour celui qui lit (« machine SM83 et gravé dans une cartouche de 32 Ko qui démarre sur une »). La console l'ignore.
`// console d'origine.`	Un commentaire : une phrase pour celui qui lit (« console d'origine. »). La console l'ignore.
`//   node gb3.mjs exemples/bonjour.cpp`	Un commentaire : une phrase pour celui qui lit (« node gb3.mjs exemples/bonjour.cpp »). La console l'ignore.
`#include <texte>    // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`#include <bouton>   // lit un bouton de la manette`	Demande à la console « bouton » (ce que c'est : lit un bouton de la manette). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lit un bouton de la manette ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`texte(6, 4, "BONJOUR");`	Appel de la fonction texte de la console, avec les arguments 6, 4, "BONJOUR". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`texte(3, 8, "APPUIE SUR A");`	Appel de la fonction texte de la console, avec les arguments 3, 8, "APPUIE SUR A". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`uint8_t compte = 0;`	Une variable : une boîte appelée compte, qui retient un nombre de 0 à 255, et qui commence à 0.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image(); // attend l'image suivante : soixante fois par seconde`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante : soixante fois par seconde ».)
`if (bouton(A)) {`	Une décision : si le bouton A est enfoncé, on fait ce qui est entre les accolades. Sinon, on passe.

`texte(3, 8, "BRAVO ");`	Appel de la fonction texte de la console, avec les arguments 3, 8, "BRAVO ". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`compte++;`	compte augmente de 1.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`if (bouton(B)) {`	Une décision : si le bouton B est enfoncé, on fait ce qui est entre les accolades. Sinon, on passe.
`texte(3, 8, "APPUIE SUR A");`	Appel de la fonction texte de la console, avec les arguments 3, 8, "APPUIE SUR A". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`return 0;`	La fonction s'arrête ici et rend 0 à celui qui l'a appelée.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

Étape 1 — rassembler

Un programme peut tenir sur plusieurs fichiers. Une ligne `#include "dessins.cpp"` veut dire « verse ici le texte du fichier dessins.cpp ». La première étape fait ces versements, pour obtenir **un seul texte**. (C'est le fichier `compilateur/inclusion.js`.)

bonjour.cpp n'a qu'un fichier : le texte rassemblé fait 34 lignes, et les lignes `#include <texte>` (avec des chevrons) ne versent rien : elles **demandent** une fonction de la console, et le compilateur s'en souvient pour la suite.

Étape 2 — analyser : du texte à l'arbre

L'analyseur lit le texte mot à mot (`compilateur/analyseur.js`). Il reconnaît les mots du langage (`int`, `while`, `if`…), les noms (`compte`), les nombres, les textes entre guillemets, les signes — et il vérifie l'orthographe du C++ : une parenthèse oubliée, un point-virgule manquant, et il s'arrête en disant la ligne.

Il range le programme en arbre, comme un arbre généalogique : le programme contient la fonction main, qui contient des instructions, qui contiennent des appels, qui contiennent des valeurs. Voici le début de main, tel que l'analyseur l'a rangé (abrégé) :


```
[
  {
    "genre": "expression",
    "valeur": {
      "genre": "appel",
      "nom": "texte",
      "arguments": [
        {
          "genre": "nombre",
          "valeur": 6,
          "ligne": 14
        },
        {
```

```
          "genre": "nombre",
          "valeur": 4,
          "ligne": 14
        },
        {
          "genre": "texte",
          "valeur": "BONJOUR",
          "ligne": 14
        }
      ]
    }
  },
  {
    "genre": "expression",
    "valeur": {
      "genre": "appel",
      "nom": "texte",
      "arguments": [
        {
          "genre": "nombre",
          "valeur": 3,
          "ligne": 15
        },
        {
          "genre": "nombre",
          "valeur": 8,
          "ligne": 15
```

Comment le lire : chaque `{ … }` est un nœud de l'arbre ; `genre` dit ce que c'est (un appel, une boucle, une variable…), et les autres champs ses morceaux : `nom` le nom de la fonction appelée, `arguments` ce qu'on lui donne, `valeur` un nombre ou un texte, `ligne` la ligne du programme d'où vient le morceau — c'est grâce à elle qu'une erreur peut dire « ligne 14 ». Ici, la ligne 14, `texte(6, 4, "BONJOUR");`, est devenue un appel à « texte » avec trois arguments : les nombres 6 et 4, et le texte BONJOUR.

L'ordinateur travaille bien mieux sur cet arbre que sur le texte : tout y est déjà rangé et nommé.

Étape 3 — compiler : de l'arbre aux instructions

L'émetteur parcourt l'arbre (`compilateur/emetteur.js`) et, pour chaque nœud, écrit les **instructions du processeur** de la Game Boy (il s'appelle SM83). Ces instructions sont très simples : « mets ce nombre dans une case », « ajoute », « saute là-bas », « appelle cette routine ». Une seule ligne de C++ en donne souvent plusieurs.

Le résultat pour bonjour.cpp : 561 octets de programme, 1 variable. Voici le début de main, relu par le désassembleur (`desassembleur.js`) — l'adresse, les octets, l'instruction, et ce qu'elle veut dire :

```
main() :
  $0181  21 86 98   ld hl, $9886
  $0184  11 56 03   ld de, Texte_2
  $0187  06 07      ld b, 7
  $0189  cd 0d 02   call ecrire_18
  $018c  21 03 99   ld hl, $9903
  $018f  11 5d 03   ld de, Texte_3
  $0192  06 0c      ld b, 12
  $0194  cd 0d 02   call ecrire_18
  $0197  af         xor a
  $0198  e0 80      ldh [main.compte], a

tantque_4 :
  $019a  cd d5 01   call AttendreImage    ; image()
```

```
  $019d  cd 71 02   call LireManette      ; bouton(…)
  $01a0  cb 67      bit 4, a
  $01a2  ca b5 01   jp z, finsi_7
  $01a5  21 03 99   ld hl, $9903
  $01a8  11 69 03   ld de, Texte_8
  $01ab  06 0c      ld b, 12
  $01ad  cd 0d 02   call ecrire_18
  $01b0  f0 80      ldh a, [main.compte]
  $01b2  3c         inc a
  $01b3  e0 80      ldh [main.compte], a

finsi_7 :
  $01b5  cd 71 02   call LireManette      ; bouton(…)
  $01b8  cb 6f      bit 5, a
  $01ba  ca c8 01   jp z, finsi_10
  $01bd  21 03 99   ld hl, $9903
  $01c0  11 75 03   ld de, Texte_11
```

Comment le lire, colonne par colonne : d'abord l'**adresse** (`$0181`…), écrite en hexadécimal — un nombre avec les chiffres 0-9 et A-F — : la place de l'instruction dans la cartouche. Puis les **octets**, la traduction exacte en nombres. Puis l'**instruction**, en assembleur. Enfin, après « ; », ce qu'elle veut dire en C++, quand on le sait.

Les quatre premières lignes sont `texte(6, 4, "BONJOUR");` : `ld hl, $9886` range dans la case « hl » l'adresse de la case de l'écran (colonne 6, ligne 4) ; `ld de, Texte_2` range l'adresse du mot BONJOUR, gravé plus loin dans la cartouche ; `ld b, 7` sa longueur, sept lettres ; `call ecrire_…` appelle la routine qui recopie les lettres à l'écran. Une ligne de C++, quatre instructions.

Les mots de l'assembleur : `ld` « charger » (ranger une valeur quelque part), `call` « appeler une routine » (puis revenir), `jp` / `jr` « sauter » ailleurs, `inc` « ajouter 1 », `xor a` « mettre a à zéro », `bit` « tester un bit ». Les noms comme `AttendreImage` ou `LireManette` sont des **routines de la console** : du code tout fait, ajouté parce que le programme a demandé `image()` et `bouton()`. `tantque_4` est le début de la boucle `while`, et `finsi_7` la fin d'un `if` : quand le bouton A n'est pas enfoncé, `jp z, finsi_7` saute par-dessus ce qu'il y a dedans.

`main.compte` **est la variable** `compte` : le compilateur lui a donné une adresse dans la mémoire, et `inc a` entre deux `ldh` est le `compte++;` du programme.

Étape 4 — fabriquer la cartouche

Une cartouche Game Boy a une forme imposée (`compilateur/cartouche.js`) : à l'adresse `$0100`, un saut vers le programme ; de `$0104` à `$0133`, le logo de Nintendo (la console refuse de démarrer sans lui) ; à `$0134`, le titre du jeu ; puis quelques réglages, et une **somme de contrôle** qui vérifie que rien n'est abîmé. Le programme commence ensuite, en `$0150`.

Voici les premiers octets de la cartouche de bonjour, en hexadécimal. Le titre lu dans l'en-tête : « BONJOUR » ; taille : 32 Ko.

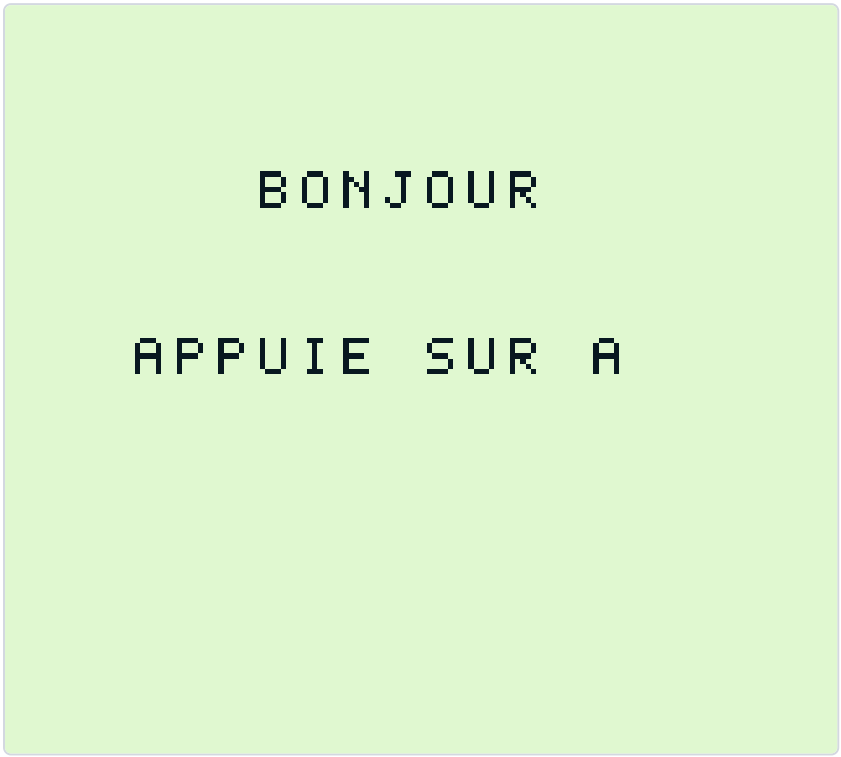
```
$0100  00 c3 50 01 ce ed 66 66 cc 0d 00 0b 03 73 00 83
$0110  00 0c 00 0d 00 08 11 1f 88 89 00 0e dc cc 6e e6
$0120  dd dd d9 99 bb bb 67 63 6e 0e ec cc dd dc 99 9f
$0130  bb b9 33 3e 42 4f 4e 4a 4f 55 52 00 00 00 00 00
$0140  00 00 00 00 00 00 00 03 00 02 00 00 00 c3 d6 5a
```

Les dessins (les lettres de la police, les tuiles du programme) sont gravés après le programme ; la cartouche est complétée jusqu'à 32 768 octets. Dans l'atelier, le bouton « ⬇ .gb » télécharge exactement ce fichier : il marcherait sur une vraie Game Boy.

Étape 5 — jouer : l'émulateur

L'émulateur imite la console, pièce par pièce (`emulateur.js`) : le processeur (qui exécute les instructions une à une), la mémoire, l'écran (qui dessine 144 lignes, 60 fois par seconde), la manette et le son. Il ne « devine » rien : il fait exactement ce que ferait la vraie machine avec les mêmes octets.

Voici l'écran de bonjour, après 30 images (une demi-seconde), photographié dans l'émulateur :



exemples/bonjour.cpp, dans l'émulateur du projet

> **À retenir**
>
> Le texte devient un arbre, l'arbre devient des instructions, les instructions deviennent une cartouche, la cartouche devient un écran. Chaque étape est un fichier du projet, et chacune vérifie son travail : si quelque chose ne va pas, c'est l'étape la plus proche de ton texte qui le dit, avec la ligne.

C. Le compilateur, fichier par fichier

Le compilateur est un dossier de fichiers JavaScript, chacun chargé d'un morceau du travail. Tu n'as pas besoin de les lire pour programmer des jeux ; mais si tu veux comprendre ou changer la console elle-même, voici où chercher. La description vient de l'en-tête de chaque fichier.

`compilateur/agrandir.js`	AGRANDIR UN DESSIN — on le dessine une fois, en petit, et l'on choisit	183 lignes
`compilateur/analyseur.js`	Lit du C++ et en fait un arbre.	936 lignes
`compilateur/cartouche.js`	Assemble les 32 Ko d'une cartouche autour du code compilé.	125 lignes
`compilateur/cartouches.js`	LA cartouche d'un programme — Game Boy OU Game Boy Color, jamais les deux.	41 lignes
`compilateur/consoles.js`	Les deux consoles — et rien d'autre.	75 lignes
`compilateur/emetteur.js`	L'arbre devient du code machine.	7791 lignes
`compilateur/inclusion.js`	Écrire un programme sur plusieurs fichiers.	237 lignes
`compilateur/inclusions-auto.js`	Écrire les « #include » qu'un programme emploie sans les avoir écrits.	122 lignes
`compilateur/png.mjs`	Écrit une image PNG, sans dépendance.	66 lignes
`compilateur/police.js`	La police du compilateur : 43 dessins de 8 × 8, en 2 bits par pixel.	411 lignes
`compilateur/source-gravee.js`	Le programme C++ d'origine, gravé DANS la cartouche.	199 lignes

Les plus importants : `analyseur.js` (le texte → l'arbre), `emetteur.js` (l'arbre → les instructions, avec toutes les fonctions de la console), `inclusion.js` (les `#include`, et la liste `BIBLIOTHEQUES` de tout ce qu'on peut demander), `cartouche.js` (la cartouche), `agrandir.js` (l'agrandissement d'un dessin, pour la taille en dernier argument).

Pour ajouter ta propre fonction à la console, un document entier l'explique pas à pas : `documents/ajouter-un-include.pdf`.

D. L’atelier : la page où l’on travaille

L’atelier est une page web : `index.html`. Pour l’ouvrir, on double-clique sur `lancer.bat` : il démarre un petit **serveur** sur ton ordinateur (avec Node.js), puis ouvre la page dans le navigateur. (Ouvrir index.html directement d’un double-clic ne marche pas : le navigateur interdit alors à la page de lire les fichiers du projet, et la page le dit.)

Ce qu’on voit : à gauche le **programme** (un éditeur de texte qui colore le code et propose les fonctions pendant qu’on tape), à droite la **console** (l’émulateur) avec ses boutons — Pause, Reset, Son, Plein écran, Capture, ⬇ .gb pour télécharger la cartouche.

Les modes, en haut de la page :

- 🎨 **CRÉATION** : écrire le programme, et des ateliers à la souris — ▦ Les tuiles (dessiner, avec 📐 Agrandir…), ♪ Les airs (la musique), 🎨 Les couleurs, 🗺 La carte (le décor), 🖼 Les modèles (dessins tout faits : formes, images, personnages) ;
- 📚 **APPRENDRE** : le parcours, leçon par leçon ; chaque leçon met son programme dans l’éditeur et le lance ;
- 🔎 **MACHINE** : les instructions que la console exécute vraiment (le résultat de l’étape 3) ;
- 🔗 **LES SOURCES** : tous les documents, dont celui-ci.

Les projets : « ✦ Nouveau » crée un dossier dans `projets/`, « 📂 Ouvrir » montre les projets du disque. Chaque compilation enregistre le projet ouvert.

Le code de la page est surtout dans `index.html` (près de 7 000 lignes, avec beaucoup de commentaires), aidé de modules : `editeur-code.js` (l’éditeur), `editeur-tuiles.js` (le dessin), `editeur-airs.js` (la musique), `projets.js` (les projets), `recherche.js` (la recherche)…

E. Les leçons et le parcours

Les 688 leçons sont écrites dans des fichiers JavaScript, dans le dossier `tuto/` : `lecons.js` (les leçons du parcours), `tutoriels.js` (une leçon par fonction de la console), `programmation.js` (le cours de programmation), `formes.js` (la série 2) et `images.js` (la série 3). `parcours.js` les range dans l'ordre, chapitre par chapitre.

Une leçon est une fiche, avec des cases qui ont un nom : son titre, son idée, ses paragraphes d'explication, son programme, ce qu'on doit voir à l'écran, et un **contrôle** — un petit programme qui vérifie que la leçon fait bien ce qu'elle promet. Voici la fiche de la leçon 2.01, abrégée :

```
{
  "titre": "Le rond",
  "numero": "2.01",
  "idee": "2.01 : le rond — un cercle. Un dessin de 16 × 16 posé au milieu de l'écran avec sprite16(), son nom
dessous.",
  "texte": [
    "…"
  ],
  "code": "// ---- 2.01 Le rond ----\n// Le rond de 16 × 16 au milieu de l'écran, et son nom dessous.\n\n#include
<texte>                    // texte() : écrire un mot à l'écran\n#include <sprite16>                //
sprite16() : poser un personnage de 16 × 16\n#include <Perso>                    // Perso : un dessin de 16 ×
16\n…",
  "aVoir": "Le rond au milieu de l'écran, et ROND écrit dessous.",
  "controle": "(c) => { … }"
}
```

Le contrôle est la garantie : `verification/verifier-tuto.mjs` compile chaque leçon, la fait tourner dans l'émulateur, et lance son contrôle (par exemple : « le lutin 0 est-il bien en (72, 56) ? »). Si une leçon ne marche plus, la vérification le dit tout de suite.

Les PDF des leçons sont fabriqués, pas écrits à la main : `outils/livret.mjs` (un livret par leçon, et le tutoriel), `outils/cours-complet.mjs` (le cours complet), `outils/cours.mjs` (le cours de programmation). Ils compilent chaque programme et photographient l'écran : une image dans un PDF est toujours le vrai écran.

F. Les vérifications : comment on sait que tout marche

Une vérification est un programme qui en contrôle un autre. Elle fait ce que ferait une personne patiente : elle compile un programme, le fait tourner dans l'émulateur, appuie sur les boutons, lit l'écran — et compare avec ce qui est attendu. Chaque contrôle s'affiche avec ✓ (bon) ou ✗ (pas bon).

Pour toutes les lancer d'un coup, dans une fenêtre de commande ouverte dans le dossier du projet :

```
npm run verifier
```

Pour en lancer une seule, par exemple celle des jeux :

```
node verification/verifier-jeux.mjs
```

Elles sont dans le dossier `verification/`. Voici chacune, avec la première phrase de sa description :

`verifier-airs.mjs`	Les airs jouent-ils vraiment ce qui est écrit ?
`verifier-atelier-airs.mjs`	L'atelier des airs tient-il ses promesses ?
`verifier-atelier.mjs`	L'atelier de tuiles tient-il ses promesses ?
`verifier-console.mjs`	Le matériel de la console répond-il vraiment ?
`verifier-couleur.mjs`	La couleur arrive-t-elle vraiment à l'écran ?
`verifier-cours.mjs`	Le cours « Apprendre à programmer » : chaque leçon compile-t-elle, et
`verifier-desassembleur.mjs`	Le désassembleur relit-il vraiment ce que le compilateur a écrit ?
`verifier-disponible.mjs`	L'atelier est-il disponible, QUELLES QUE SOIENT les conditions ?
`verifier-exemples.mjs`	Les exemples tournent-ils vraiment sur la console ?
`verifier-export.mjs`	L'export HTML tourne-t-il vraiment tout seul, sans l'atelier ?
`verifier-frappe.mjs`	Ce qu'on tape est-il bien ce qui arrive dans le programme ?
`verifier-inclusion.mjs`	Écrire sur plusieurs fichiers change-t-il quelque chose à la cartouche ?
`verifier-jeux.mjs`	Les quatre jeux de la série 3 se jouent-ils jusqu'au bout ?
`verifier-langage.mjs`	Le langage fait-il ce qu'il dit ?
`verifier-marche.mjs`	La marche joue-t-elle vraiment ?
`verifier-methodes.mjs`	Les méthodes font-elles ce qu'elles disent ?
`verifier-page.mjs`	La page tient-elle ses promesses ?
`verifier-portage.mjs`	Le portage depuis gameboy2 change-t-il quoi que ce soit à l'écran ?
`verifier-projets.mjs`	Les projets sur le disque : le service tient-il, et refuse-t-il ce qu'il doit ?
`verifier-recherche.mjs`	Les recherches marchent-elles, partout ?
`verifier-refus.mjs`	Ce qui doit être refusé l'est-il, et le message dit-il quoi faire ?
`verifier-source-gravee.mjs`	Le programme C++ gravé dans la cartouche revient-il TEL QUEL ?
`verifier-surplace.mjs`	Un dessin écrit là où on le pose arrive-t-il vraiment à l'écran ?
`verifier-tout-le-jeu.mjs`	L'onglet « 📦 Tout le jeu » tient-il ses promesses ?
`verifier-tuto-page.mjs`	La page du tutoriel tient-elle debout ?

`verifier-tuto.mjs`	Chaque leçon du tutoriel compile-t-elle, et fait-elle ce qu'elle promet ?
`verifier-tutoriels.mjs`	`TUTORIELS.md` dit-il encore la vérité ?

> **Pourquoi c'est important**
>
> Un projet de cette taille ne peut pas être vérifié à la main à chaque changement. Les vérifications le font en quelques minutes, à chaque fois, sans se lasser : quand on change le compilateur, elles disent tout de suite si une leçon, un jeu ou une page est cassé.

G. Ajouter quelque chose : un jeu, une leçon, une image, une fonction

Un jeu

1. Écris le programme dans l'atelier (ou dans un fichier `exemples/mon-jeu.cpp`) et vérifie qu'il compile et qu'il marche.

2. Pour qu'il soit dans **le menu des exemples** : dans `index.html`, ajoute une ligne `<option value="monjeu">MON JEU — …</option>` dans la liste `id="exemples"`, et une ligne `monjeu: 'exemples/mon-jeu.cpp',` dans la liste `EXEMPLES`.

3. Pour qu'il soit dans 📂 **Ouvrir** : crée le dossier `projets/mon_jeu/` avec `principal.cpp` (le programme), `projet.json` (son titre), et la cartouche. Le plus simple : ✦ Nouveau dans l'atelier, puis colle ton programme.

4. Ajoute une partie jouée dans `verification/verifier-jeux.mjs`, pour qu'on sache toujours qu'il marche.

Une leçon

1. Dans le fichier de la série qui convient (`tuto/lecons.js`, `tuto/formes.js`, `tuto/images.js`…), ajoute une fiche : titre, idée, texte, code, ce qu'on doit voir, et un contrôle.

2. Lance `node verification/verifier-tuto.mjs` : ta leçon est compilée et contrôlée avec toutes les autres.

3. Régénère les documents : `node outils/tutoriels.mjs`, puis les PDF (`node outils/livret.mjs`, `node outils/cours-complet.mjs`).

Une image

1. Dans `bibliotheque.js`, ajoute un dessin à la liste `IMAGES` : un numéro, un nom en MAJUSCULES, un titre, et 16 rangées de 16 signes (`.`, `-`, `+`, `#`).

2. Elle apparaît aussitôt dans 🖼 Les modèles, et une leçon de la série 3 est fabriquée pour elle.

Une fonction de la console

Tout est expliqué pas à pas, du premier mot au dernier, dans `documents/ajouter-un-include.pdf` : où écrire la fonction, comment la rendre disponible avec `#include`, comment l'aide de l'éditeur la propose, et comment la vérifier.

H. Enregistrer et partager : git

git garde l'historique du projet, comme un album de photos : chaque **commit** est une photo du projet entier, avec une phrase qui dit ce qui a changé. On peut toujours revenir à une photo d'avant — rien n'est jamais perdu.

GitHub est un site qui garde une copie du projet sur internet. **Pousser** (push), c'est envoyer ses nouvelles photos là-bas : le projet est alors sauvegardé, et d'autres peuvent le voir.

Les trois commandes de tous les jours, dans une fenêtre de commande ouverte dans le dossier :

```
git status                          # ce qui a changé depuis la dernière photo
git add -A                          # tout préparer pour la photo
git commit -m "Ce que j'ai changé"  # prendre la photo, avec sa phrase
git push                            # l'envoyer sur GitHub
```

Ce qui n'est jamais envoyé est écrit dans le fichier `.gitignore` : par exemple le dossier `cartouches/`, qui contient des jeux du commerce (ils appartiennent à Nintendo, on n'a pas le droit de les partager).

L'historique de ce qui a changé, en phrases : `documents/CHANGEMENTS.md`, mis à jour à chaque étape.

Le lexique : chaque mot, expliqué simplement

Les 76 mots techniques qu'on rencontre dans ce document, par ordre alphabétique.

accolades { }	Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
affectation (=)	« x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
analyseur	La première partie du compilateur : il lit le texte et en fait un arbre (compilateur/analyseur.js).
arbre (syntaxique)	Ce que l'analyseur fabrique : le programme rangé comme un arbre généalogique — le programme contient des fonctions, qui contiennent des instructions, qui contiennent des valeurs.
argument	Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
assembleur	La langue du processeur, écrite en mots courts (ld, call, jp…) : chaque mot est une instruction de la machine. Le compilateur traduit le C++ en assembleur, puis en octets.
bloc	Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
booléen (vrai / faux)	Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
boucle	Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
boucle du jeu	« while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
bouton()	Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
break	Sort tout de suite de la boucle (ou du « switch ») où l'on est.
C++	Le langage dans lequel ces programmes sont écrits. Le compilateur de l'atelier en comprend une partie, adaptée à la Game Boy.
cartouche	Le boîtier qui contient un jeu ; ici, le fichier .gb (ou .gbc en couleur) que le compilateur fabrique : 32 768 octets.
chaîne de caractères (texte)	Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
commentaire (//)	Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
commit (git)	Une « photo » enregistrée du projet, avec un message qui dit ce qui a changé. On peut toujours revenir à une photo d'avant.
compilateur	Le traducteur qui transforme le texte du programme en octets que la console comprend, et qui signale les erreurs.

condition	Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
console	La machine qui joue : ici, la Game Boy (ou Game Boy Color), imitée dans le navigateur par l'émulateur.
constante (const)	Une valeur qui a un nom et ne change jamais : « const uint8_t VIES = 3; ».
continue	Passe directement au tour suivant de la boucle.
coordonnées	Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
couleur, palette	En couleur (Game Boy Color), chaque dessin a quatre teintes ; une palette dit quelles couleurs. couleurFond() les règle pour le décor, couleurLutin() pour les lutins.
défilement	Le décor qui glisse quand le héros avance : defiler(x, y) décale tout le fond.
dossier	Un classeur dans l'ordinateur, qui contient des fichiers (et d'autres dossiers).
else (sinon)	Ce que l'on fait quand la condition du « if » est fausse.
émetteur	La partie du compilateur qui transforme l'arbre en instructions du processeur (compilateur/emetteur.js).
émulateur	Un programme qui imite une machine : ici, une Game Boy dans le navigateur.
fichier	Un document enregistré dans l'ordinateur : un texte, une image, une cartouche… Son nom finit par une extension (.cpp, .js, .gb…) qui dit son genre.
fonction	Un groupe d'instructions qui a un nom, qu'on lance en écrivant son nom et des parenthèses. Celles de la console (texte, poser…) sont toutes prêtes ; on peut écrire les siennes.
for	Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
git	Un programme qui garde l'historique du projet, photo après photo, et qui l'envoie sur internet (GitHub) pour le partager et le sauvegarder.
hasard()	Une fonction de la console qui rend un nombre au hasard, de 0 à 255.
hexadécimal	Une façon d'écrire les nombres avec 16 chiffres : 0 à 9 puis A à F. « $FF » veut dire 255 ; « $0150 » veut dire 336. Pratique pour les octets : un octet s'écrit toujours en deux chiffres.
HTML	Le langage des pages web : l'atelier (index.html) est une page HTML, que le navigateur affiche.
if (si)	Teste une condition : si elle est vraie, on fait le bloc qui suit.
image()	Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
#include	En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
incrémenter (++)	« x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.

indice	Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
instruction	Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
JavaScript (.js)	Le langage dans lequel l'atelier, le compilateur et l'émulateur sont écrits. Le navigateur sait l'exécuter.
lutin (sprite)	Un dessin qui bouge au pixel près, par-dessus le décor : le héros, un ennemi. sprite() en place un de 8 × 8, sprite16() un de 16 × 16. La console en a 40.
main	« int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
mémoire (RAM)	Là où la console range ses variables pendant le jeu : 8 Ko de cases, effacées quand on l'éteint.
module	Un fichier .js qui fait une seule chose et qui la prête aux autres (avec « export » et « import »).
navigateur	Le programme qui affiche les pages web : Chrome, Edge, Firefox… L'atelier s'ouvre dedans.
Node.js	Un programme qui exécute du JavaScript en dehors du navigateur. Il fait tourner le petit serveur de l'atelier, les outils et les vérifications.
octet	Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
opérateurs de calcul	+ plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
opérateurs de comparaison	== égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
opérateurs logiques	&& « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
paramètre	Dans la définition d'une fonction, la boîte qui reçoit une valeur donnée à l'appel : dans « void f(uint8_t x) », x est un paramètre.
Perso	Un dessin de 16 × 16 pixels (ou 32 × 32) : quatre tuiles en carré (ou seize). « Perso HEROS = { … }; ».
pixel	Un tout petit carré de couleur ; l'écran en a 160 de large et 144 de haut.
point-virgule (;)	Il termine une instruction, comme le point termine une phrase.
poser()	Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
processeur	La puce qui exécute les instructions, une à une. Celle de la Game Boy s'appelle SM83 et fait environ 4 millions de pas par seconde.
programme	La liste d'instructions qu'on écrit, et que le compilateur traduit en cartouche.
push (git)	Envoyer ses commits sur internet (GitHub) : le projet y est alors sauvegardé et partagé.
registre	Une toute petite case DANS le processeur (a, b, c, h, l…), où il fait ses calculs. Beaucoup plus rapide que la mémoire.

return	Termine la fonction et rend une valeur à celui qui l'a appelée.
ROM	La mémoire de la cartouche, qu'on lit sans pouvoir l'écrire : le programme, les dessins, les textes.
serveur (local)	Un petit programme qui « sert » les pages de l'atelier au navigateur. lancer.bat le démarre ; il ne sort pas de ton ordinateur.
struct	Un modèle qui regroupe plusieurs boîtes sous un nom : « struct Ennemi { uint8_t x; uint8_t y; }; ».
switch / case	Choisir entre plusieurs cas selon la valeur d'une variable, au lieu d'une suite de « if ».
tableau	Une rangée de boîtes sous un même nom : « uint8_t cases[10]; » donne cases[0] à cases[9].
taille (agrandir)	Un nombre en dernier argument de sprite, sprite16, sprite32, poser… : le dessin, écrit une fois, montré autant de fois plus grand.
texte()	Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
Tuile	Un dessin de 8 × 8 pixels, écrit rangée par rangée avec des signes (. - + #). Le décor en est fait.
uint8_t	Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
variable	Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
vérification	Un programme qui contrôle qu'un autre marche : il compile, fait tourner, appuie sur les boutons, et compare avec ce qui est attendu. ✓ veut dire « bon », ✗ « pas bon ».
void	Devant une fonction : elle ne rend rien, elle agit seulement.
while (tant que)	Répète un bloc tant qu'une condition est vraie.